

Top 50 Git Commands

BASICS (Version Control Operations)

1. `git init`
Initializes a new Git repository in the current folder.
2. `git clone <repo-url>`
Clones a remote Git repository into your local machine.
3. `git status`
Shows the state of the working directory and staging area.
4. `git add <file>`
Stages a file for commit.
5. `git add .`
Stages all changes (new, modified, deleted files).
6. `git commit -m "message"`
Commits the staged changes with a message.
7. `git commit -am "message"`
Adds and commits tracked files in one command.
8. `git log`
Shows the commit history.
9. `git diff`
Shows changes between the working directory and the last commit.
10. `git diff --staged`
Shows changes between staged files and last commit.

BRANCHING & MERGING

11. `git branch`
Lists all branches.
 12. `git branch <branch-name>`
Creates a new branch.
 13. `git checkout <branch-name>`
Switches to a specific branch.
 14. `git switch <branch-name>`
Newer way to switch branches.
 15. `git switch -c <new-branch>`
Creates and switches to a new branch.
 16. `git merge <branch>`
Merges another branch into the current branch.
 17. `git branch -d <branch>`
Deletes a local branch.
 18. `git branch -D <branch>`
Force deletes a local branch.
 19. `git merge --abort`
Cancels a merge if there are conflicts.
 20. `git stash`
Temporarily stores changes in working directory.
-

REMOTE REPOSITORY MANAGEMENT

21. `git remote -v`
Shows URLs of remotes (like origin).
22. `git remote add origin <url>`
Links local repo to a remote.

23. `git push -u origin main`
Pushes changes and sets the upstream.
 24. `git push`
Pushes commits to the remote branch.
 25. `git pull`
Fetches and merges remote changes.
 26. `git fetch`
Downloads updates from remote but doesn't merge.
 27. `git push origin <branch-name>`
Pushes a specific branch to remote.
 28. `git push --force`
Forces a push (used carefully!).
 29. `git push origin --delete <branch>`
Deletes a remote branch.
 30. `git pull origin <branch>`
Pulls changes from specific remote branch.
-

RESET, REVERT, CLEANUP

31. `git reset <file>`
Unstages a file.
32. `git reset --hard`
Resets working directory and index to last commit.
33. `git reset --soft HEAD~1`
Moves HEAD back but keeps changes staged.
34. `git revert <commit>`
Creates a new commit that undoes a previous one.

- 35. `git clean -f`
Deletes untracked files.
 - 36. `git clean -fd`
Deletes untracked files and directories.
-

TAGS, LOGS, HISTORY

- 37. `git tag`
Lists all tags.
 - 38. `git tag <tag-name>`
Creates a tag.
 - 39. `git tag -a v1.0 -m "Version 1"`
Creates an annotated tag.
 - 40. `git show <tag>`
Shows tag details and associated commit.
 - 41. `git log --oneline`
Shows simplified commit history.
 - 42. `git log --graph`
Displays commits as a graph.
-

TOOLS & CONFIGURATION

- 43. `git config --global user.name "Your Name"`
Sets global username.
- 44. `git config --global user.email "you@example.com"`
Sets global email.
- 45. `git config --list`
Lists current configurations.

46. `git blame <file>`

Shows who changed which line of a file.

47. `git cherry-pick <commit>`

Applies a commit from another branch.

48. `git rebase <branch>`

Reapplies commits on top of another base tip.

49. `git reflog`

Shows the history of branch movements.

50. `git archive`

Creates a zip/tar archive of the repository content.